

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 6

DEEP LEARNING INTRO — SPRINT 1

Neural Networks, Backpropagation & Training with TensorFlow / PyTorch

The first sprint of the Phase 3 capstone: how a neural network is built and trained. Interns learn network architecture, activation functions, forward/backpropagation, and optimizers — then build and train their first network in TensorFlow/Keras and run the full Sprint 1 review cycle on their own project.

PHASE 3 — SPRINT 1 • 40 HOURS • 5 TRAINING DAYS • HANDS-ON

Prepared by BinX Tech
Palestine | Nablus

BinX
TECH

1. Week 6 Overview

Week	Week 6 of 10 — Phase 3: Deep Learning & Applied Project, Sprint 1
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 11-12 combined)
Format	On-site / Remote / Hybrid — notebooks in Jupyter/Colab, committed to GitHub via feature branches
Focus	Neural networks & deep learning intro: architecture, activations, backpropagation, optimizers, batch normalization; TensorFlow/Keras; Sprint 1 of the Phase 3 capstone project
Prerequisite	Weeks 1-5 completed; Phase 3 project selected and Sprint 1 planned in Week 5
Mentor Supervision	Mentor Code & Notebook Review mid-sprint (Day 3) via pull request; Sprint Review + Retrospective on Day 5

2. Sprint 1 Structure

Phase 3 runs across 4 one-week sprints. Week 6 is Sprint 1, following the program's standard sprint cycle:

Event	Description
Sprint Planning (Day 1)	Confirm the Sprint 1 goal from Week 5, select backlog tasks (dataset, EDA, baseline), and commit to the sprint goal.
Daily Stand-up (Daily)	3-minute update: what was completed, what is next, any blockers (data issues, model errors).
Mentor Code & Notebook Review (Day 3)	Mentor reviews the notebook and code via GitHub pull request; structured comments on methodology and results.
Sprint Review (Day 5)	Demo completed pipeline stages / model results. Incomplete tasks move to Sprint 2 with documented reasons.
Sprint Retrospective (Day 5)	What went well, what to improve, and one specific action for Sprint 2.

3. Week 6 Learning Objectives

- Complete Sprint 1 planning and establish a baseline model for the Phase 3 project.
- Explain the architecture of a neural network: neurons, layers, weights, and biases.
- Explain activation functions and choose the right one for hidden and output layers.
- Describe forward propagation, loss, backpropagation, and gradient descent as one training loop.
- Build, compile, train, and evaluate a neural network in TensorFlow/Keras.
- Run the full Sprint 1 cycle: planning, stand-ups, mentor review, Sprint Review, and Retrospective.

4. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Sprint 1 planning & baseline; neural network architecture (neurons, layers, weights)
Day 2	8 hrs	Activation functions; forward propagation and loss
Day 3	8 hrs	Backpropagation, gradient descent, and optimizers; Mentor Code & Notebook Review
Day 4	8 hrs	Building & training a network in TensorFlow/Keras; batch normalization & dropout
Day 5	8 hrs	Tuning & evaluating the network; Sprint Review & Retrospective

5. Day-by-Day Curriculum

Day 1 — Sprint 1 Planning & Neural Network Architecture — 8 hours

Learning Objectives

- Complete Sprint 1 planning and establish a baseline model for the project.
- Explain a single neuron as a weighted sum, bias, and activation.
- Describe the role of input, hidden, and output layers and what 'deep' means.

Key Topics

- Sprint 1 planning: goal, backlog, and baseline first
- Why deep learning: unstructured, high-dimensional data
- The neuron: weighted sum + bias + activation (the Week 2 dot product)
- Layers: input, hidden, output
- Weights and biases as the learned parameters

Lesson Content

1.1 Sprint 1 Planning & the Baseline First

Day 1 opens with formal Sprint Planning: confirm the Sprint 1 goal set in Week 5, break it into backlog tasks (dataset finalization, EDA, and a baseline model), and commit to the goal. Sprint 1's real objective is to establish a baseline — a simple model (from Weeks 3-4) whose score every later model, including this week's neural network, must beat. Without a baseline, a neural network's score is meaningless.

1.2 Why Deep Learning

Classical ML (Weeks 3-4) works well on structured, tabular data. Deep learning shines where classical methods struggle: images, text, audio, and other high-dimensional, unstructured data with complex patterns. A neural network learns its own features directly from raw data, which is why it powers modern computer vision and NLP — the topics of Weeks 7-8.

1.3 The Neuron

A single neuron is the building block, and it is exactly the operation from Week 2: it takes inputs, multiplies each by a weight, sums them, adds a bias, then passes the result through an activation function. In one line: $\text{output} = \text{activation}(\text{dot}(\text{inputs}, \text{weights}) + \text{bias})$. A neural network is just many of these neurons connected in layers.

```
# One neuron, conceptually:  
# z = (x1*w1) + (x2*w2) + ... + bias      <- the dot product from Week 2  
# output = activation(z)
```

1.4 Layers and Architecture

Neurons are organized into layers, and data flows through them in order:

Layer	Role
Input layer	Receives the raw features (one node per feature)
Hidden layer(s)	Learn intermediate representations; more/wider layers = more capacity
Output layer	Produces the final prediction (shape depends on the task)

"Deep" learning simply means a network with multiple hidden layers. Each layer transforms its input and passes the result forward, and stacking layers lets the network learn increasingly abstract patterns.

1.5 Weights and Biases Are What's Learned

The weights and biases are the network's parameters — the numbers adjusted during training. A fresh network starts with random weights and makes random predictions; training is the process of nudging these weights until the predictions become accurate. Everything on Days 2-3 is about how that nudging works.

The neuron is not new math — it is the Week 2 dot product plus a bias plus an activation. A neural network is that same operation, stacked and repeated. Keeping this in mind demystifies deep learning.

Hands-On Lab: Sprint 1 Kickoff & Baseline

- Step 1: Complete Sprint 1 planning: confirm the goal and select the first backlog tasks.
- Step 2: Finalize the project dataset and complete a brief EDA in the project notebook.
- Step 3: Train a simple baseline model (e.g. logistic/linear regression from Week 3) and record its metric.
- Step 4: Commit the baseline notebook to a feature branch and open a draft pull request.
- Step 5: In Markdown, state the baseline score every later model must beat.

Tools Used

Scikit-learn (baseline) • Pandas • Jupyter/Colab • Git & GitHub

Day 2 — Activations, Forward Propagation & Loss — 8 hours

Learning Objectives

- Explain why non-linear activations are essential.
- Choose the correct activation for hidden and output layers.
- Describe forward propagation and select the right loss function for a task.

Key Topics

- Why activations introduce non-linearity
- Common activations: ReLU, sigmoid, softmax, tanh
- Choosing activations by layer and task
- Forward propagation: computing a prediction
- Loss functions: MSE, binary/categorical cross-entropy

Lesson Content

2.1 Why Activation Functions Matter

Without an activation function, a neural network — no matter how many layers — collapses into a single linear model, because a stack of linear operations is still linear. The activation function introduces non-linearity, which is what lets the network learn complex, curved patterns. It is the single ingredient that makes deep learning powerful.

2.2 Common Activation Functions

Function	Output Range	Use It For
ReLU	0 to $+\infty$ (negatives $\rightarrow$ 0)	Hidden layers — the default choice, fast and effective
Sigmoid	0 to 1	Output layer for binary classification (a probability)

Softmax	0 to 1, sums to 1	Output layer for multi-class classification (class probabilities)
Tanh	-1 to +1	Hidden layers when zero-centered output helps

The practical rule: use ReLU in hidden layers by default, and choose the output activation by the task — sigmoid for binary, softmax for multi-class, and none (linear) for regression.

2.3 Forward Propagation

Forward propagation is the network making a prediction: data enters the input layer and flows forward, each layer computing its weighted sums and activations, until the output layer produces a prediction. It is the network running "forward" to answer — exactly what happens at prediction time, and the first half of every training step.

```
# Forward pass, conceptually:
# layer1 = ReLU(dot(X, W1) + b1)
# layer2 = ReLU(dot(layer1, W2) + b2)
# output = sigmoid(dot(layer2, W3) + b3)
```

2.4 The Loss Function

After a forward pass, the loss function measures how wrong the prediction was by comparing it to the true label. A high loss means bad predictions; the entire goal of training is to make the loss as small as possible. The right loss depends on the task:

Task	Loss Function
Regression	Mean Squared Error (MSE)
Binary classification	Binary cross-entropy
Multi-class classification	Categorical cross-entropy

Hands-On Lab: Activations & the Forward Pass

- Step 1: Plot ReLU, sigmoid, and tanh over a range of inputs to see how each transforms values.
- Step 2: For the Phase 3 project task, decide the correct output activation and loss function, and justify both.
- Step 3: By hand or in NumPy, compute one forward pass for a tiny 2-layer network on a sample input.
- Step 4: Document the choices and the forward-pass result in Markdown.

Tools Used

NumPy • Matplotlib • Jupyter/Colab

Day 3 — Backpropagation, Gradient Descent & Optimizers — 8 hours

Learning Objectives

- Describe the four-step training loop of a neural network.
- Explain gradient descent and the role of the learning rate.
- Explain backpropagation conceptually and name common optimizers, epochs, and batches.

Key Topics

- The training loop: forward → loss → backprop → update
- Gradient descent and the loss surface
- The learning rate and its effect
- Backpropagation: assigning blame via the chain rule

- Optimizers (Adam, SGD), epochs, and batches

Lesson Content

3.1 The Training Loop in One Picture

Training a neural network is a loop that repeats over the data: (1) forward pass to make predictions, (2) compute the loss, (3) backpropagation to find how each weight contributed to the loss, (4) update the weights to reduce the loss. Repeat thousands of times, and the random initial weights gradually become accurate ones.

3.2 Gradient Descent

Gradient descent is the update step. The gradient tells you the direction of steepest increase in the loss for each weight; gradient descent moves each weight in the opposite direction — downhill — to reduce the loss. The intuition: you are standing on a foggy hillside (the loss surface) and take small steps in the steepest downhill direction until you reach a valley (minimum loss).

3.3 The Learning Rate

The learning rate controls the step size in gradient descent, and it is the most important hyperparameter in deep learning:

Learning Rate	Effect
Too high	Overshoots the minimum; loss bounces or diverges
Too low	Trains very slowly; may get stuck
Just right	Steady, efficient decrease in loss

3.4 Backpropagation

Backpropagation is the algorithm that computes the gradients efficiently. It works backward from the loss through the network, using the chain rule of calculus to determine exactly how much each weight contributed to the error. You will not implement it by hand — frameworks compute it automatically (PyTorch's autograd, TensorFlow's GradientTape) — but understanding that it is 'assigning blame' to each weight is essential.

3.5 Optimizers, Epochs, and Batches

Term	Meaning
Optimizer	The algorithm that applies gradient descent; Adam is the strong default
Epoch	One full pass through the entire training dataset
Batch	A subset of data processed before each weight update (e.g. 32 samples)
SGD vs. Adam	SGD is basic gradient descent; Adam adapts the step size per weight

You rarely tune gradient descent by hand: use the Adam optimizer as a reliable default, start with a learning rate around 0.001, and adjust only if the loss curve misbehaves.

Hands-On Lab: Understanding Training + Mentor Review

- Step 1: Draw or describe the four-step training loop in a Markdown cell for the project notebook.
- Step 2: Experiment with a tiny network at three learning rates (too high, too low, good) and plot the loss curves.
- Step 3: Explain, in your own words, what backpropagation computes and why the chain rule is involved.

- Step 4: Open a pull request with the current project notebook for the mid-sprint Mentor Code & Notebook Review and address the feedback.

Tools Used

NumPy / TensorFlow • Matplotlib • Git & GitHub (pull request)

Day 4 — Building & Training a Network in Keras — 8 hours

Learning Objectives

- Build a neural network with the Keras Sequential API.
- Compile, train, and evaluate the network, and read its training history.
- Apply batch normalization and dropout to stabilize training and reduce overfitting.

Key Topics

- TensorFlow/Keras as the high-level framework
- The Sequential API and Dense layers
- Compile / fit / evaluate workflow
- Reading the training history (loss curves)
- Batch normalization and dropout

Lesson Content

4.1 TensorFlow / Keras

You will not implement neural networks from scratch in production — frameworks handle the math. TensorFlow with its Keras API is the standard high-level framework: you describe the architecture layer by layer, and Keras handles forward propagation, backpropagation, and optimization automatically. (PyTorch is the main alternative, covered as needed; both are on the program's stack.)

4.2 Building a Model with the Sequential API

The Keras Sequential API stacks layers in order. A Dense (fully connected) layer connects every input to every neuron. This example builds a network for binary classification:

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(64, activation="relu", input_shape=(n_features,)),
    Dense(32, activation="relu"),
    Dense(1, activation="sigmoid"), # binary output
])
```

4.3 Compile, Train, Evaluate

The three-step Keras workflow mirrors the Scikit-learn API from Week 3. `compile()` sets the optimizer, loss, and metrics; `fit()` runs the training loop; `evaluate()` measures performance on the test set.

```
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])
history = model.fit(X_train, y_train,
                   validation_data=(X_val, y_val),
                   epochs=50, batch_size=32)
model.evaluate(X_test, y_test)
```

The history object records the loss and metric per epoch for both training and validation — plotting it is how you diagnose training, using the exact overfit/underfit reasoning from Week 4.

4.4 Batch Normalization and Dropout

Two standard layers improve training and fight overfitting. Batch normalization normalizes each layer's inputs during training, making the network train faster and more stably. Dropout randomly "switches off" a fraction of neurons during each training step, forcing the network not to rely too heavily on any one path — a powerful regularizer, the deep-learning equivalent of the regularization from Week 4.

```
from tensorflow.keras.layers import BatchNormalization, Dropout
model = Sequential([
    Dense(64, activation="relu", input_shape=(n_features,)),
    BatchNormalization(),
    Dropout(0.3),                # drop 30% of neurons each step
    Dense(32, activation="relu"),
    Dense(1, activation="sigmoid"),
])
```

Hands-On Lab: Training a Neural Network

- Step 1: Build a Keras Sequential network appropriate for the Phase 3 project task (correct output layer and loss).
- Step 2: Compile with Adam and the right loss, then train with a validation split for at least 30 epochs.
- Step 3: Plot training vs. validation loss and accuracy from the history object and diagnose the fit.
- Step 4: Add dropout and/or batch normalization and compare the new loss curves to the previous run.
- Step 5: Evaluate on the test set and compare the score to the Day 1 baseline.

Tools Used

TensorFlow / Keras • Matplotlib • Jupyter/Colab (GPU)

Day 5 — Tuning, Evaluation & Sprint Review — 8 hours

Learning Objectives

- Tune a neural network systematically, one variable at a time.
- Use EarlyStopping and checkpoints to train efficiently and keep the best model.
- Complete the full Sprint 1 review and retrospective cycle.

Key Topics

- Tuning priorities: learning rate, size, dropout, batch size
- Callbacks: EarlyStopping and ModelCheckpoint
- Assembling proof for Sprint Review (loss curves, metric table)
- Per-task acceptance criteria
- Sprint Review and Retrospective

Lesson Content

5.1 Tuning a Neural Network

Neural networks have many knobs. The highest-impact ones to tune, in rough priority: learning rate, network width/depth (neurons and layers), dropout rate, and batch size. Change one thing at a time and watch the validation loss curve — the same disciplined, one-variable-at-a-time approach used for hyperparameter tuning in Week 4.

5.2 Callbacks: EarlyStopping and Checkpoints

Keras callbacks automate good training practice. EarlyStopping halts training once the validation loss stops improving, preventing wasted epochs and overfitting. ModelCheckpoint saves the best model seen during training, so you keep the best version even if later epochs get worse.

```
from tensorflow.keras.callbacks import EarlyStopping
es = EarlyStopping(monitor="val_loss", patience=5,
                  restore_best_weights=True)
model.fit(X_train, y_train, validation_data=(X_val, y_val),
          epochs=100, callbacks=[es])
```

5.3 Preparing for Sprint Review

Every result this sprint — the baseline score, the neural network's architecture and score, and the tuning experiments — needs clear evidence (loss curves, the metric table, notebook narrative) before it can be demoed. Per the program's acceptance criteria: the notebook runs without errors, code is committed to the correct branch, the pull request is approved by the mentor, results are documented in Markdown, and metrics are logged and compared to the baseline.

5.4 Sprint Review & Retrospective

Sprint Review: demo the completed Sprint 1 work to the mentor — the baseline and the first trained neural network with its evaluation. Any unfinished backlog task moves to Sprint 2 with a documented reason. The Retrospective closes the sprint honestly: what went well, what to improve, and one concrete change for Sprint 2 (e.g. "log every experiment's config in MLflow from the start").

Hands-On Lab: Sprint 1 Close-Out

- Step 1: Tune the network by changing one hyperparameter at a time and record each run's validation score.
- Step 2: Add EarlyStopping and confirm training halts at the right point, keeping the best weights.
- Step 3: Assemble the Sprint 1 evidence: baseline vs. neural network scores, architecture, and loss curves.
- Step 4: Ensure all Sprint 1 work is committed, the pull request is merged after mentor approval, and the notebook is documented.
- Step 5: Present the Sprint Review, then write the Retrospective with one concrete change for Sprint 2.

Tools Used

TensorFlow / Keras • Matplotlib • Git & GitHub • Draft project report

6. Week 6 Deliverables

By the end of Week 6 (Sprint 1), every intern must submit the following to their mentor and GitHub repository:

- The completed Sprint 1 plan (goal, backlog) and a trained baseline model with its recorded metric.
- An activations & forward-pass notebook with plotted activation functions and a justified output/loss choice.
- A training-mechanics notebook: the training loop explained, learning-rate experiments, and loss curves.
- A trained Keras neural network with compile/fit/evaluate, dropout/batch-norm, and diagnosed loss curves.
- Tuning experiments with EarlyStopping, and a metric table comparing the network to the Day 1 baseline.
- All Sprint 1 work committed to GitHub via reviewed pull requests, plus the written Sprint Retrospective.

7. Week 6 Evaluation Criteria

Scored by the assigned mentor at the Sprint 1 review using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Understanding of DL concepts	Runs code but unclear on how training works	Explains architecture, activations, and training clearly	Deep understanding; connects backprop and tuning to results
TensorFlow/Keras proficiency	Builds a network with heavy guidance	Builds, trains, and evaluates networks independently	Fluent; applies dropout/batch-norm and callbacks effectively
Model evaluation & baseline	Trains a network but no baseline comparison	Diagnoses fit and compares to a baseline	Rigorous evaluation; tuned network clearly beats baseline
Sprint discipline	Misses stand-ups or planning steps	Completes full sprint cycle: plan, stand-ups, review, retro	Proactive sprint management; flags blockers early
Notebook & Git workflow	Sporadic commits, weak narrative	Regular commits, PR review, clear Markdown	Consistent, descriptive, reproducible, well-organized
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

8. Technical Stack — Week 6

Deep Learning Framework	TensorFlow / Keras (Sequential, Dense, Dropout, BatchNormalization)
Training	Adam / SGD optimizers, callbacks (EarlyStopping, ModelCheckpoint)
Baseline & Data	Scikit-learn, Pandas, NumPy
Compute	Google Colab (free GPU/TPU)
Environment	Jupyter/Colab, Matplotlib, Git & GitHub (feature branches + pull requests)

9. Notes & Best Practices

A neural network is the Week 2 dot product stacked and repeated — keep that in mind and the framework code stops being magic. Two rules carry over from earlier weeks and matter more than ever here: always compare against the baseline, and always read the loss curves before trusting a result. The neural network is one tool in the Phase 3 project, not the whole project — it only earns its place if it beats the simpler baseline.